

GUIA FOR DUMMIES

Próximos Passos FootStock

100% funcional · 50.000 usuários · Web + iOS + Android · Apple Store + Google Play · Ionic + Capacitor

Este guia explica em linguagem simples tudo que precisa ser feito para o FootStock estar completamente no ar: com login real, banco de dados preparado para 50k usuários, app instalável no celular e publicado nas lojas da Apple e do Google.

Gerado em 24/03/2026 · Fun4Fans do Brasil Ltda

O Roteiro Completo

FASE 1	Colocar o App Web no Ar (Pré-requisito de tudo)
1.1	Configurar o Vite e conectar o frontend ao backend
1.2	Deploy no Railway (backend) + Cloudflare Pages (frontend)
1.3	Testar login, preços e painel admin em produção
FASE 2	Login e Banco de Dados Reais
2.1	Migrations e seed no banco de produção
2.2	Testar criação de conta, login e logout
2.3	Configurar recuperação de senha por e-mail
FASE 3	Preparar para 50.000 Usuários
3.1	Redis Adapter para múltiplas instâncias do servidor
3.2	Worker Thread para o motor de mercado
3.3	PgBouncer para pool de conexões do banco
3.4	Escalar para Fly.io com auto-scaling
FASE 4	Transformar em App Mobile com Ionic + Capacitor
4.1	O que é Ionic e por que é a escolha certa
4.2	Instalar Ionic + Capacitor no projeto
4.3	Configurar para iOS e Android
4.4	Testar no celular físico
FASE 5	Publicar na Apple Store
5.1	Criar conta de desenvolvedor Apple (US\$99/ano)
5.2	Configurar certificados e provisioning profiles
5.3	Preparar os assets (ícones, screenshots)
5.4	Submeter para revisão da Apple
FASE 6	Publicar na Google Play
6.1	Criar conta de desenvolvedor Google (US\$25 único)
6.2	Gerar o APK/AAB assinado
6.3	Preparar a ficha da loja
6.4	Publicar e aguardar aprovação
FASE 7	Pagamentos e Assinaturas
7.1	Integrar Pagar.me (cartão + PIX)
7.2	In-App Purchase (Apple) e Google Play Billing

FASE 8 Checklist Final — App 100% Pronto

1

FASE 1 — App Web no Ar

Pré-requisito para tudo — sem isso, nada mais funciona

Clique no título para mais detalhes neste guia

Esta fase é a base de tudo. Antes de pensar em mobile, lojas ou 50k usuários, o app precisa estar rodando na internet — não só no seu computador.

Passo 1.1 Configurar o Vite e conectar o frontend ao backend

O Vite é o programa que 'empacota' o código React em arquivos que o navegador entende. O freelancer vai fazer isso.

O que o freelancer vai fazer:

```
# Dentro da pasta FootStock/  
$ npm create vite@latest footstock-web -- --template react  
$ cd footstock-web  
$ npm install  
  
# Copiar o JSX para dentro do projeto Vite  
$ cp ../App/FootStockApp\ \ (7\).jsx src/App.jsx  
  
# Configurar a URL do backend no arquivo .env do Vite  
VITE_API_URL=https://api.seudominio.com.br  
VITE_WS_URL=https://api.seudominio.com.br  
  
# Testar localmente  
$ npm run dev  
→ Abre em http://localhost:5173
```



O freelancer vai aplicar os patches de integração que já foram gerados (FootStockApp_backend_patches.js). São 8 substituições documentadas — leva menos de 1 hora.

Passo 1.2 Deploy: Railway (backend) + Cloudflare Pages (frontend)

O freelancer vai colocar os dois na internet seguindo o Guia de Deploy que já foi gerado. Ao final, você terá:

URL	O que é	Como acessar
https://api.seudominio.com.br	Backend (servidor)	Só APIs — não abre no navegador dir
https://app.seudominio.com.br	App do usuário	Qualquer pessoa com o link acessa
https://admin.seudominio.com.br	Painel admin	Só você, com senha de admin

Passo 1.3 **Testar tudo em produção**

Teste	Como fazer	Resultado esperado
Backend funcionando	Abrir: <code>api.seudominio.com.br/health</code>	<code>{"ok":true}</code>
Login funcionando	Criar conta no app → fazer login	Entra na tela principal
Preços ao vivo	Ver a tela de Mercado	Preços atualizando a cada 3s
Admin funcionando	Login <code>admin@footstock.com/admin123</code>	Painel admin abre
Banco persistindo	Criar conta, fechar browser, abrir de novo	Conta ainda existe



Lembre de trocar a senha `admin123` ANTES de divulgar o app. Qualquer pessoa que souber essa senha terá acesso total.

2

FASE 2 — Login e Banco de Dados Reais

Sistema de autenticação completo funcionando

Clique no título para mais detalhes neste guia

Após a Fase 1, o login já funciona — mas precisamos garantir que está seguro, robusto e com recuperação de senha.

Passo 2.1 Migrations e seed no banco de produção

O freelancer vai rodar estes comandos no servidor Railway:

```
# No terminal remoto do Railway:
$ npx prisma migrate deploy
■ Applied 1 migration

$ node prisma/seed.js
■ 40 clubes inseridos
■ admin@footstock.com criado
■ Seed concluído!
```

■ ■ O seed já foi gerado e está pronto. O banco vai ter os 40 clubes, preços calculados e os usuários admin criados automaticamente.

Passo 2.2 Testar criação de conta e login

Após o deploy, teste você mesmo:

Teste	O que fazer
Criar conta	Abrir o app → Criar conta com seu e-mail pessoal
Fazer login	Sair e entrar novamente com as credenciais criadas
Verificar banco	Abrir Adminer (localhost:8080 ou Railway) — ver tabela User
Trocar senha admin	Login com admin@footstock.com → Perfil → Trocar senha

Passo 2.3 Recuperação de senha por e-mail

O freelancer vai adicionar a rota POST `/api/auth/forgot-password` ao servidor. Ela envia um e-mail com link de reset. Usar o serviço Resend (gratuito até 3k e-mails/mês).

Serviço	Custo	O que faz
Resend	Grátis (3k/mês) · Pro: US\$20/mês	Envia e-mails transacionais (reset de senha, boas-v

SendGrid	Grátis (100/dia) · Essentials: US\$20/mês	Alternativa popular — mais configuração
Amazon SES	US\$0,10 por 1k e-mails	Mais barato em escala — requer conta AWS



Recomendação: Resend. Integração com Node.js em 3 linhas de código e o plano gratuito é suficiente para os primeiros meses.

3

FASE 3 — Preparar para 50.000 Usuários

3 mudanças cirúrgicas que mudam tudo

Clique no título para mais detalhes neste guia

Sem essas 3 mudanças, o servidor vai travar quando tiver muita gente. Com elas, o FootStock aguenta 50k usuários simultâneos.



Esta fase é técnica — o freelancer executa, você não precisa entender o detalhe.

O importante é que seja feita ANTES do lançamento.

Passo 3.1

Redis Adapter — sincronizar múltiplas instâncias

O que é: quando o servidor cresce e vira 2, 3 ou 10 instâncias, elas precisam se comunicar. O Redis Adapter faz isso automaticamente.

O freelancer adiciona 2 linhas no server.js:

```
# npm install @socket.io/redis-adapter
import { createAdapter } from '@socket.io/redis-adapter';
io.adapter(createAdapter(pubClient, subClient));
```

Resultado: 10 instâncias funcionam como 1



Sem isso: se você tiver 2 instâncias do servidor, usuário A conectado na instância 1 não recebe os preços transmitidos pela instância 2.

Passo 3.2

Worker Thread — isolar o motor de mercado

O que é: o motor calcula preços de 40 ativos a cada 2,5s. Esse cálculo não pode travar as respostas das APIs. O Worker Thread move o motor para um processo separado.

```
# O freelancer cria: backend/motor.worker.js
# Move marketTick() para lá
# Motor roda em thread separada — API nunca para

→ API responde em <50ms mesmo durante tick do motor
```



Sem isso: com 1000+ usuários conectados, o motor pode travar as respostas da API por até 200ms em cada tick — usuários percebem lentidão.

Passo 3.3

PgBouncer — pool de conexões do banco

O que é: cada instância do servidor abre várias conexões com o PostgreSQL. Sem controle, isso satura o banco. O PgBouncer gerencia um pool de conexões compartilhado.

```
# Só mudar o DATABASE_URL no .env:  
DATABASE_URL="postgresql://...@pgbouncer:6432/footstock?pgbouncer=true"
```

- De N×10 conexões para máximo 20 conexões totais
- Banco aguenta 10× mais carga

- Railway já oferece PgBouncer integrado — o freelancer só precisa ativar nas configurações do banco e trocar a URL.

Passo 3.4 Escalar para Fly.io com auto-scaling

Quando ultrapassar 2k usuários simultâneos, migrar o backend para Fly.io que sobe instâncias automaticamente conforme a demanda.

Estágio	Usuários	Custo/mês	Ação necessária
E1	até 2k	R\$80–150	Railway simples — já está assim
E2	2k–15k	R\$1.000–1.500	Fly.io + Redis Adapter + Worker Thread
E3	15k–50k	R\$2.700–4.000	Fly.io auto-scale + PgBouncer + Redis Cloud

4

FASE 4 — App Mobile com Ionic + Capacitor

O mesmo código React vira iOS + Android

Clique no título para mais detalhes neste guia

O que é o Ionic?

Ionic é um framework que pega o seu app React existente e o transforma num aplicativo instalável no celular — tanto iOS quanto Android. Você não precisa reescrever nada. O Capacitor (parte do Ionic) cria uma casca nativa que envolve o app web.

Comparação	React Native	Ionic + Capacitor	Flutter
Reescrever o código	Sim (70%+)	Não (0%)	Sim (100%)
Performance	■■■■■	■■■■■	■■■■■
Curva de aprendizado	Alta	Baixa	Alta
Tempo de integração	2–4 semanas	1–2 dias	4–8 semanas
Custo para o FootStock	Alto	Baixo	Alto

Ionic + Capacitor é a escolha certa para o FootStock: aproveita 100% do código React, suporta WebSocket, biometria (WebAuthn) e tem integração direta com Apple Store e Google Play.

Passo 4.1 Instalar Ionic + Capacitor

O freelancer vai executar dentro da pasta do projeto Vite:

```
# Instalar Capacitor
$ npm install @capacitor/core @capacitor/cli
$ npx cap init FootStock com.fun4fans.footstock

# Adicionar plataformas iOS e Android
$ npm install @capacitor/ios @capacitor/android
$ npx cap add ios
$ npx cap add android

# Plugins nativos úteis
$ npm install @capacitor/splash-screen
$ npm install @capacitor/status-bar
$ npm install @capacitor/push-notifications
```

Passo 4.2 Configurar capacitor.config.ts

O arquivo de configuração do Capacitor define o nome, ID do app e a URL do servidor:

```
// capacitor.config.ts
const config: CapacitorConfig = {
  appId: "com.fun4fans.footstock",
  appName: "FootStock",
  webDir: "dist", // pasta gerada pelo Vite
  server: {
    url: "https://app.seudominio.com.br",
    cleartext: true,
    androidScheme: 'https',
  }
};
```

Passo 4.3 Build e sincronizar com iOS/Android

```
# 1. Gerar o build web (pasta dist/)
$ npm run build

# 2. Copiar o build para as plataformas nativas
$ npx cap sync

# 3. Abrir no Xcode (iOS) ou Android Studio
$ npx cap open ios
$ npx cap open android
```

Passo 4.4 Testar no celular físico

Antes de publicar nas lojas, testar no celular é obrigatório:

Plataforma	Como testar	O que verificar
Android	Habilitar modo desenvolvedor no celular → conectar por USB no Android Studio	Layout, ícones, fontes, cores, biometria
iOS	Conta Apple Developer gratuita + conectar iPhone no Xcode	Desbloqueio com Face ID/Touch ID



O teste no iOS exige um Mac. Se o freelancer não tiver Mac, o serviço Codemagic (codemagic.io) faz o build na nuvem por ~US\$10/build.

5

FASE 5 — Publicar na Apple Store

O processo mais burocrático — mas totalmente possível

Clique no título para mais detalhes neste guia



A Apple tem o processo de aprovação mais rigoroso do mercado. Tempo médio de revisão: 1–3 dias úteis. Rejeições são comuns na primeira tentativa — não se preocupe.

Passo 5.1

Criar conta de desenvolvedor Apple

Item	Detalhe
URL	developer.apple.com/programs/enroll/
Custo	US\$99/ano (pago no cartão de crédito)
Tipo de conta	Individual (você mesmo) ou Organization (empresa — Fun4Fans)
Recomendação	Organization — aparece na App Store como 'Fun4Fans do Brasil Ltda'
Tempo	1–3 dias para aprovação da conta
Necessário	Apple ID + cartão de crédito + DUNS number (empresa)



DUNS number é um código de identificação empresarial internacional. Solicite gratuitamente em dnb.com/duns-number.html — leva 1–5 dias úteis.

Passo 5.2

Preparar os assets obrigatórios

A Apple exige materiais visuais específicos. O freelancer (ou um designer) vai criar:

Asset	Tamanho	Observação
Ícone do app	1024×1024 px PNG	Sem transparência · sem arredondamento (Apple faz)
Screenshots iPhone	1290×2796 px	Mínimo 3, máximo 10 imagens da tela real do app
Screenshots iPad	2048×2732 px	Obrigatório se o app suportar iPad
Preview video	Opcional	Vídeo de até 30s mostrando o app funcionando
Ícone de notificação	40×40, 60×60 px	Versões menores do ícone principal

Passo 5.3

Configurar certificados e provisioning

Esta é a parte mais técnica — o freelancer faz tudo isso no Xcode:

Passo	O que é	Quem faz
-------	---------	----------

Bundle ID	Identificador único: com.fun4fans.footstock	Freelancer no App Store Connect
Certificado	Arquivo que 'assina' o app digitalmente	Xcode gera automaticamente
Provisioning Profile	Vincula o app à sua conta Apple	Xcode gera automaticamente
Capabilities	Permissões: câmera, Face ID, notificações	Freelancer no Xcode

Passo 5.4 Submeter para revisão

Passo	Onde	O que preencher
1. App Store Connect	appstoreconnect.apple.com	Criar o app, preencher nome, categoria, descrição
2. Classificação App Store	App Store Connect	FootStock: 4+ (sem conteúdo adulto)
3. Política de privacidade	Wikipédia	Criar página simples ou usar termsfeed.com
4. Upload do bundle	Xcode → Archive → Upload	Freelancer faz pelo Xcode ou Transporter
5. Submeter	App Store Connect	Botão 'Submit for Review'
6. Aguardar	E-mail Apple	1–3 dias úteis para resposta



Motivos comuns de rejeição: (1) app não funciona offline sem internet, (2) ausência de política de privacidade, (3) referência a pagamentos externos sem usar Apple Pay. O freelancer vai cuidar disso.

6

FASE 6 — Publicar na Google Play

Processo mais simples que a Apple — aprovação em horas

Clique no título para mais detalhes neste guia

- A Google Play tem aprovação muito mais rápida que a Apple (horas vs dias) e o processo de submissão é mais simples. Taxa única de US\$25.

Passo 6.1 Criar conta de desenvolvedor Google Play

Item	Detalhe
URL	play.google.com/console
Custo	US\$25 — taxa única (não é anual como Apple)
Conta	Conta Google normal + aceitar termos de desenvolvedor
Tipo	Organização — Fun4Fans do Brasil Ltda
Tempo	Aprovação em minutos

Passo 6.2 Gerar o APK/AAB assinado

O freelancer vai gerar o arquivo de instalação do app Android:

```
# No Android Studio:  
Build → Generate Signed Bundle / APK  
→ Escolher: Android App Bundle (AAB) — recomendado  
→ Criar keystore (arquivo de assinatura — guardar para sempre!)  
→ Build release
```

■ Gera: app-release.aab (~20MB)

- O keystore (arquivo .jks) é como a 'chave da casa' do seu app Android. Se perder, não consegue atualizar o app na Play Store. Guarde em local seguro com backup.

Passo 6.3 Preencher a ficha da loja

Campo	Conteúdo sugerido
Título	FootStock — Bolsa do Futebol
Descrição curta	Simule a bolsa de valores com cotas de clubes de futebol brasileiro
Descrição longa	Invista em cotas de 40 clubes da Série A e B, acompanhe preços em tempo real, e:

Categoria	Finanças
Classificação	Livre (todos os públicos)
Política privacidade	URL da página de política (obrigatório)

Assets necessários (mesmos da Apple podem ser reaproveitados):

Asset	Tamanho
Ícone	512×512 px PNG
Feature graphic	1024×500 px — banner da loja
Screenshots phone	No mínimo 2 imagens (1080×1920 px recomendado)
Screenshots tablet	Opcional mas recomendado

Passo 6.4 Publicar e aguardar aprovação

Passo	Ação
1. Upload	Play Console → Criar app → Produção → Fazer upload do AAB
2. Revisão	Google analisa automaticamente em 2–24 horas
3. Publicação	App aparece na Play Store após aprovação
4. Atualizar	Gerar novo AAB com versão incrementada e fazer novo upload



Diferente da Apple, a Google raramente rejeita apps na primeira submissão. O processo todo (do upload à publicação) costuma levar menos de 24 horas.

7

FASE 7 — Pagamentos e Assinaturas

Como receber pelo app em cada plataforma

Clique no título para mais detalhes neste guia



ATENÇÃO: A Apple obriga que pagamentos de assinatura dentro do app iOS usem o sistema deles (30% de taxa). Isso impacta o preço que você cobra. Leia atentamente esta seção.

Há três cenários distintos de pagamento:

Plataforma	Sistema de pagamento	Taxa da plataforma	Recomendação
Web (navegador)	Pagar.me ou Stripe	1,5–3%	Pagar.me para Brasil (PIX)
Android (Play)	Google Play Billing OU Pagar.me externo	15–30% (Google) ou 0% (externo)	Redirecionar para web para pagamentos
iOS (App Store)	Apple In-App Purchase (obrigatório)	30% (padrão) ou 15% (pequenas empresas)	Implementar StoreKit 2 — c

Passo 7.1 Pagar.me para Web e Android

Para o app web e Android, usar Pagar.me (plataforma brasileira que aceita PIX, boleto e cartão):

Item	Detalhe
Cadastro	pagar.me — conta gratuita, aprovação em 1–2 dias
Taxa	1,39% no débito · 2,19% no crédito à vista · PIX: 0,99%
Integração	O freelancer integra via API REST — já temos a rota <code>/api/subscriptions/checkout</code>
Webhook	Pagar.me avisa o backend quando o pagamento é confirmado

Passo 7.2 Apple In-App Purchase (obrigatório para iOS)

Para o app iOS, a Apple exige que os pagamentos de assinatura usem o sistema dela. O freelancer vai implementar usando a biblioteca `@capacitor-community/apple-sign-in` e configurar os produtos no App Store Connect:

Passo	Onde	O que fazer
1. Criar produtos	App Store Connect → In-App Purchases	Criar Craque Mensal (R\$19,90) · Lenda Mensal (R\$49,90)
2. Implementar	Código com StoreKit	Freelancer integra usando <code>@capgo/capacitor-purchases</code>
3. Webhook Apple	App Store Connect → Notifications	Configurar URL do backend para receber confirmações
4. Testar	Conta de teste (Sandbox)	Apple fornece contas de teste sem cobrar



Estratégia recomendada: cobrar R\$19,90/mês no web e R\$24,90/mês no iOS (para cobrir a taxa de 30% da Apple e ainda ter margem).

8

FASE 8 — Checklist Final

App 100% pronto para lançamento

Clique no título para mais detalhes neste guia

Verifique cada item antes de anunciar o app publicamente:

■ WEB

Item	Fase
☐ App web abrindo em <code>https://app.seudominio.com.br</code>	1
☐ Admin abrindo em <code>https://admin.seudominio.com.br</code>	1
☐ Login e criação de conta funcionando	2
☐ Preços atualizando em tempo real (WebSocket)	1
☐ Ordens de compra e venda executando	1
☐ Extrato registrando transações	1
☐ Recuperação de senha por e-mail funcionando	2
☐ HTTPS em todas as URLs (cadeado verde)	1
☐ Senha admin trocada (não é mais admin123)	1

■ MOBILE

Item	Fase
☐ App instalando no Android via Google Play	6
☐ App instalando no iPhone via Apple Store	5
☐ Biometria (Face ID / Touch ID) funcionando no celular	4
☐ Notificações push funcionando	4
☐ App funcionando sem travar em celular mais antigo	4
☐ Tela de splash e ícone corretos nas lojas	5/6

■ ESCALABILIDADE

Item	Fase
☐ Redis Adapter instalado (múltiplas instâncias)	3
☐ Motor de mercado em Worker Thread	3
☐ PgBouncer ativo no banco de dados	3
☐ Fly.io configurado com auto-scaling	3
☐ Monitoramento ativo (UptimeRobot + Sentry)	3

[]	Backup automático do banco habilitado	3
-----	---------------------------------------	---

■ PAGAMENTOS

■	Item	Fase
[]	Pagar.me integrado e testado (web + Android)	7
[]	Apple In-App Purchase configurado e aprovado	7
[]	Webhooks de confirmação de pagamento funcionando	7
[]	Plano Craque e Lenda sendo ativados após pagamento	7

Resumo de custos e tempo para o app completo

Fase	O que é	Tempo	Custo freelancer	Custo plataforma
1	App web no ar	2–4 dias	R\$ 600–1.200	R\$ 80–150/mês (Railway)
2	Login e banco reais	1–2 dias	R\$ 200–400	Incluso na Fase 1
3	Escalar 50k usuários	2–3 dias	R\$ 400–800	R\$ 1.000–1.500/mês
4	Ionic + Capacitor	1–2 dias	R\$ 300–600	Gratuito
5	Apple Store	3–5 dias	R\$ 500–1.000	US\$99/ano
6	Google Play	1–2 dias	R\$ 200–400	US\$25 único
7	Pagamentos	3–5 dias	R\$ 500–1.000	1,5–3% por transação
TOTAL Tudo completo		3–6 semanas	R\$ 2.700–5.400	R\$ 1.200–1.800/mês